DESIGN AND DEVELOPMENT OF NEXUS-AI

An AI based UNIVERSAL PROBLEM SOLVING SYSTEM(UPSS)

Team-DATAFORGE

Corner Stone Project Report

Milestone 1

Submitted By

Vansh Pratap Singh Jadon (BTCS24O1145)

Faculty Mentor

Dr. Kirti Raj Bhatele

Assistant Professor

Department of Computer Science & Engineering

माधव प्रौद्योगिकी एवं विज्ञान संस्थान, ग्वालियर (म.प्र.), भारत

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR (M.P.), INDIA

Deemed University

(Declared under Distinct Category by Ministry of Education, Government of India)

NAAC ACCREDITED WITH A++ GRADE

July-Dec 2026

1. Problem Statement

Modern AI systems are generally designed to solve specific types of problems or perform isolated tasks. A user may need different AI models, tools, APIs, databases, and applications to solve a single complex problem. Existing systems also lack a unified mechanism for understanding a problem, decomposing it into tasks, dynamically selecting suitable AI agents and tools, coordinating their execution, maintaining contextual memory, and validating the final result.

Therefore, there is a need for a **universal, domain-agnostic AI platform** that can accept complex real-world problems, intelligently plan their solution, dynamically assemble the required agents and tools, execute tasks through appropriate providers, learn from results, and deliver a reliable final response.

2. Proposed Solution

Nexus-AI is a universal multi-agent intelligence platform designed to solve complex problems through **autonomous planning, dynamic agent collaboration, intelligent tool selection, memory augmentation, and result validation**.

Instead of relying on a single AI model, Nexus-AI analyzes the user's request, identifies its intent, domain, and complexity, and generates an execution plan. The system can then create an appropriate team of specialized AI agents, select the required tools and providers dynamically, execute tasks using parallel, sequential, or collaborative strategies, and evaluate the generated results through reflection and validation.

The platform also incorporates **short-term conversational memory, long-term knowledge, and RAG-based retrieval**, allowing relevant context to be used during decision-making and execution.

Core capabilities

- Universal problem understanding
- Intelligent task decomposition and planning
- Dynamic multi-agent team generation
- Agent-to-agent collaboration
- Dynamic tool and provider selection
- Parallel and sequential task execution
- Memory-augmented reasoning

- Reflection and self-evaluation
- Result validation and quality checking
- Human-in-the-loop approval when required
- Structured response generation
- Continuous memory and knowledge updates
- Modular and extensible architecture

3. Tech Stack

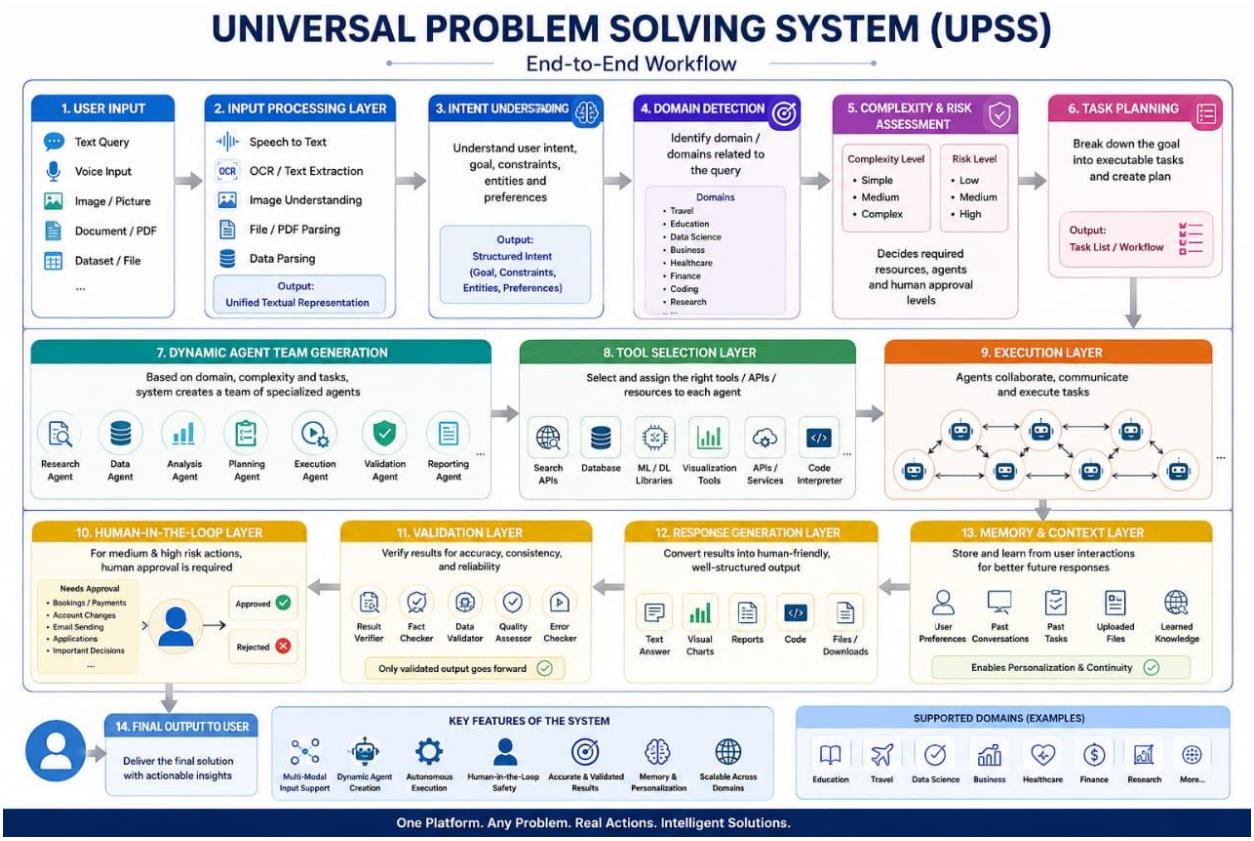
Layer	Technologies
Backend	Python, FastAPI
AI / LLM Layer	OpenAI, Google Gemini, Groq, Ollama and extensible LLM providers
Multi-Agent System	Custom Agent Architecture, Agent Collaboration, Task Orchestration
Planning	Planner, Task Decomposition, Dependency Management
Tool System	Tool Registry, Tool Discovery, Tool Manager, Tool Executor
Provider System	Provider Registry, Provider Selector, Provider Factory
Memory	Conversation Memory, Long-Term Memory, RAG
Vector Storage	ChromaDB / FAISS
Database	PostgreSQL / SQLAlchemy
Caching	Redis
Validation	Input Validation, Output Validation, Safety & Consistency Checks
Execution	Async Execution, Parallel/Sequential Strategies, Retry & Error Handling
Observability	Logging, Metrics, Execution Tracking
Containerization	Docker / Docker Compose
Data Validation	Pydantic

Layer	Technologies
Frontend	Web-based interface *(planned/extendable)*

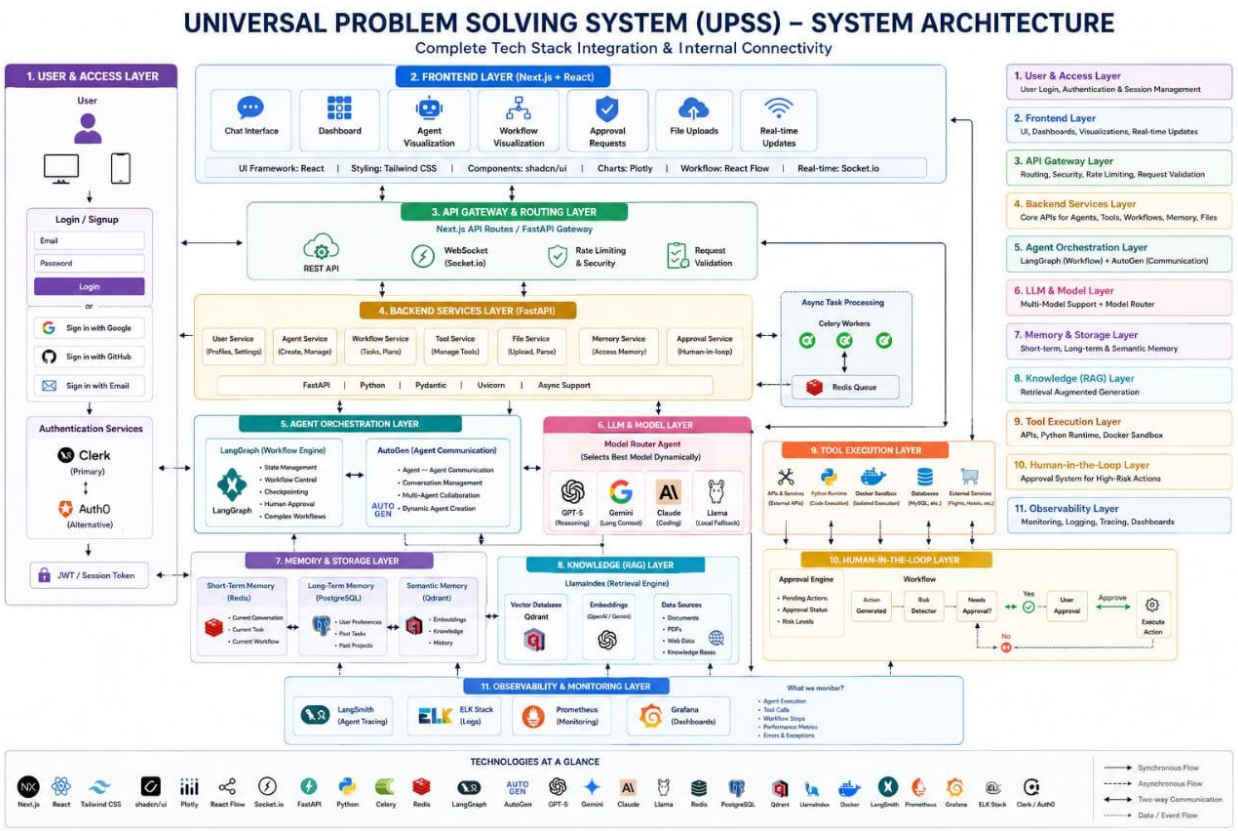
Design principles

Modular • Scalable • Domain-Agnostic • Dynamic • Extensible • Secure • Observable

4. E2E System Workflow



5. System Architecture



Architectural Highlights

1. Domain Agnostic:
The platform is designed to handle problems across different domains rather than being restricted to a single application.

2. Multi-Agent by Design:
Different agents can be assigned specialized responsibilities and collaborate to solve complex tasks.

3. Dynamic Tool Ecosystem:
Tools are discovered and registered through the Tool Registry, while capabilities and providers are used to dynamically select appropriate execution resources.

4. Provider Abstraction:
The provider layer separates tool logic from external service implementations, allowing providers to be added or replaced without redesigning the complete system.

5. Memory-Augmented Intelligence:
Relevant conversational context and stored knowledge can be retrieved during planning and execution.

6. Quality-Controlled Execution:
Reflection, validation, retries, and optional human approval provide additional control over generated results.

7. Extensible Architecture:
New agents, tools, providers, domains, and execution strategies can be added as independent modules.